

Hashing and Bloom Filters

Topics

- Hashing (Fast operations : Store data, retrieve data)
- Double hashing, Cuckoo Hashing
- HashMap in Java
- Bloom filter data structure
 - + uses hashing
 - + saves memory space

Bloom filters applications

- Bitcoin wallet app in phones use this data structure
 - wallet synchronization
filter transactions that only affect your wallet addresses.
 - memory space savings
- Weak password detection
- Cybersecurity (virus scanning)

<https://bitcoinops.org/en/topics/transaction-bloom-filtering/>

Hashing

- A set of N elements.
- Goal is to do $O(1)$ time **Add** and **Contains** query*
- Queries: Membership queries
Does this set contain x ?
- Eg: Find the frequency of occurrences of any given word in a book.

Hashing

- Uses an array (typically called a hash table) to hold the key/value pairs.

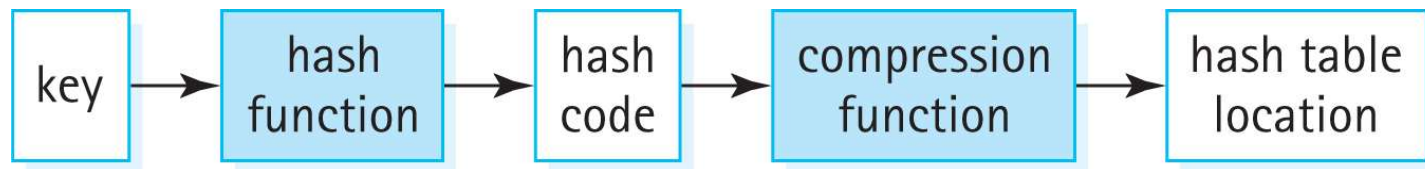
key: word

value: frequency

- Hashing involves determining array indices directly from the key of the entry being stored/accessed.

The Hash Function

- Keys might not be integers.



- Synonyms for “hash code” include “hash value,” and “hash.”

Hashing vs Binary Tree

- Balanced binary search trees can be used instead of hashing.
 $O(\log N)$ time for add/contains.
- How to approach $O(1)$ time?
- Abstractions: Python Dictionary data type
Java HashMap
Java Hashtable (thread safe)
C++ unordered_map

Use case: De-duplication

- Find and Remove duplicates

HashMap

```
import java.util.HashMap;    // import the HashMap class
```

```
HashMap<String, Integer> numbers = new HashMap<String, Integer>();  
numbers.put( "one", 1 );  
numbers.put( "two", 2 );  
numbers.put( "three", 3 )
```

HashMap

```
import java.util.HashMap;    // import the HashMap class
```

```
HashMap<String, Integer> numbers = new HashMap<String, Integer>();  
numbers.put("one", 1);  
numbers.put("two", 2);  
numbers.put("three", 3)
```

```
numbers.get("two");    // returns value i.e., 2
```

```
numbers.containsKey("ten");    // returns false
```

HashMap practice

- Create a map with keys as string and values as string to store data of the form

<u>Key</u>	<u>Value</u>
------------	--------------

Charlie	Computer Science
---------	------------------

Elon	Physics
------	---------

..	..
----	----

Hashtable

```
import java.util.Hashtable;    // import the Hashtable class
```

```
Hashtable<String, Integer> numbers = new Hashtable<String,Integer>();  
numbers.put("one", 1);  
numbers.put("two", 2);  
numbers.put("three", 3)
```

Implementation

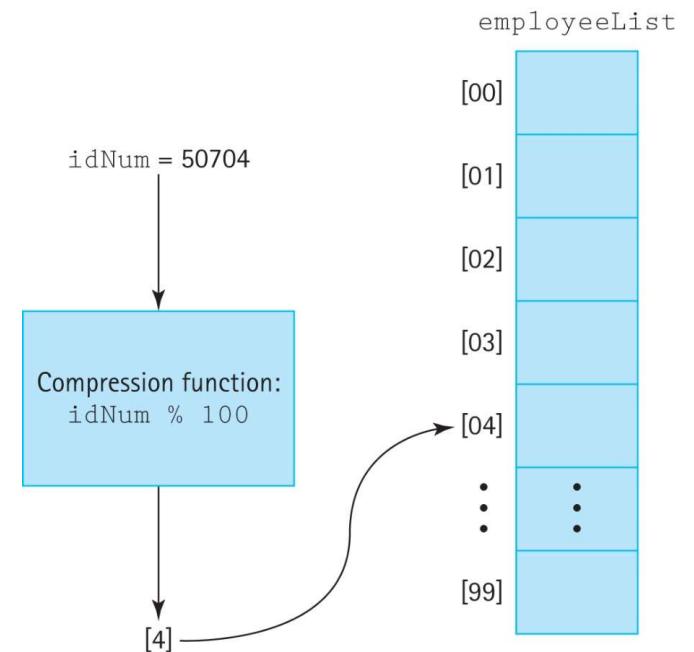
- Using arrays (open addressing)
- Using linked list
 - chaining

Hashing basic

- Input Data k
- Hash function (k) $\Rightarrow$ hash code
 $h(k)$
- Hash code is an integer used to store d in an array (table)
 $\text{int location} = h(k)$

Compression function in hashing

- If we have a positive integral key, such as an ID number, we can just use the key as the index into the array.
- If the range of key values is larger than the array we must “compress” the key into a usable index.

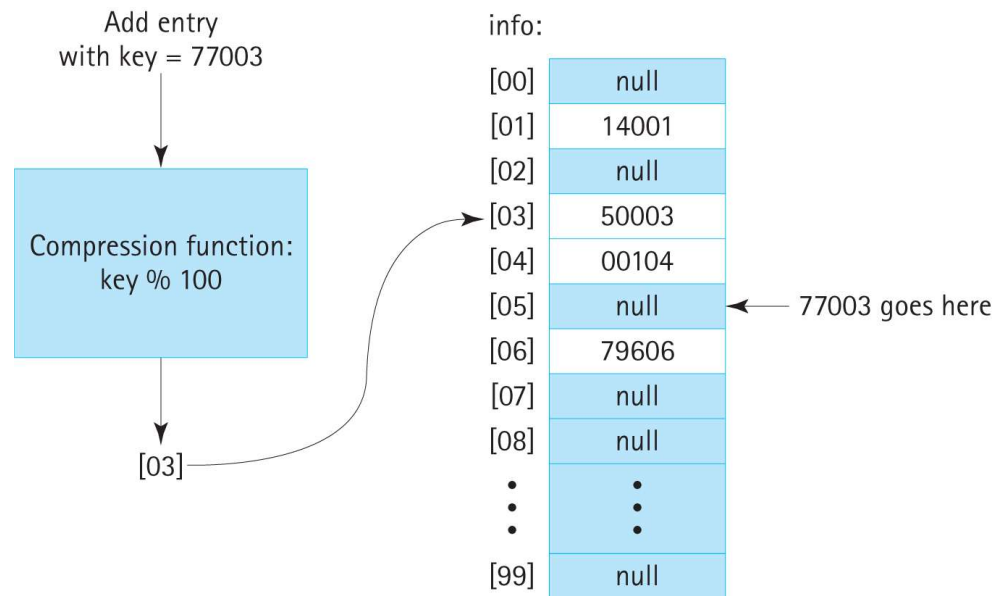


Collisions

- If two keys go to the same array location we call it a “collision.”
- Minimizing such collisions is the biggest challenge in designing a good hashing system
- Collision resolution
 - linear probing, quadratic probing, double hashing, cuckoo hashing

Collision resolution policies

- Linear probing: store the colliding entry into the next available space:



14001
53702
43201

70002
43101
99902

0	null
1	null
2	null
3	null
4	null
5	null
6	null
7	null
8	null
9	null
10	null
99	null

Insert the numbers to hash table (array)
by resolving collisions through linear
probing.

Linear Probing

Answer

Order of Insertion:	info:	
14001	[00]	null
53702	[01]	14001
43201	[02]	53702
70002	[03]	43201
43101	[04]	70002
99902	[05]	43101
	[06]	99902
	[07]	null
	[08]	null
	[09]	null
	[10]	null
	⋮	⋮
	[99]	null

(a) Linear Probing

Problem on hashing

- Input: { 1, 2, 3, 1, 2, 2, 4 }
- Calculate frequency of occurrence of each number.

1 (key) => 2 (value)

2 => 3

3 => 1

HashMap map in java

key, value => elem, count

for each element **e** in stream of input

if(map.containsKey(**e**))

 cnt = map.get(**e**)

 map.put(**e**, cnt++)

else

 map.put(**e** , 1)

Problem: 2-SUM

Input: An unsorted array A of n integers, and a target integer t .

Goal: Determine whether or not there are two numbers x, y in A satisfying $x + y = t$.

Problem: 2-SUM

Input: An unsorted array A of n integers, and a target integer t .

Goal: Determine whether or not there are two numbers x, y in A satisfying $x + y = t$.²

- Brute force approach:
 - try all pairs,
 - find sum of each pair,
 - match sum with t .

2-SUM (Attempt #1)

Input: array A of n integers and a target integer t .

Output: “yes” if $A[i] + A[j] = t$ for some $i, j \in \{1, 2, 3, \dots, n\}$, “no” otherwise.

```
for  $i = 1$  to  $n$  do
```

```
     $y := t - A[i]$ 
```

```
    if  $A$  contains  $y$  then
```

```
        // linear search
```

```
        return “yes”
```

```
return “no”
```

2-SUM (Sorted Array Solution)

Input: array A of n integers and a target integer t .

Output: “yes” if $A[i] + A[j] = t$ for some $i, j \in \{1, 2, 3, \dots, n\}$, “no” otherwise.

```
sort  $A$                                 // using a sorting subroutine
for  $i = 1$  to  $n$  do
     $y := t - A[i]$ 
    if  $A$  contains  $y$  then                // binary search
        return “yes”
return “no”
```

Quiz 12.2

What's the running time of an educated implementation of the sorted array-based algorithm for the 2-SUM problem?

- a) $\Theta(n)$
- b) $\Theta(n \log n)$
- c) $\Theta(n^{1.5})$
- d) $\Theta(n^2)$

2-SUM (Hash Table Solution)

Input: array A of n integers and a target integer t .

Output: “yes” if $A[i] + A[j] = t$ for some $i, j \in \{1, 2, 3, \dots, n\}$, “no” otherwise.

$H :=$ empty hash table

for $i = 1$ to n **do**

 INSERT $A[i]$ into H

for $i = 1$ to n **do**

$y := t - A[i]$

if H contains y **then**

 // using LOOKUP

 return “yes”

return “no”

Hashing wins over binary search

- 2-sum problem
- $O(N)$ time on average case assumption (in expectation)
 - use amortization when array size doubles
 - load factor = $\text{\#objects}/\text{arraySize}$
- Optimal

HW: 3 sum problem

Collision resolution

- Linear probing
- Double hashing
- Cuckoo hashing

Double Hashing

- Combine two hash functions h_1 and h_2

$$h(k, i) = (h_1(k) + i * h_2(k)) \% m$$

- k is key
- i is probe number = $\{0, 1, 2, \dots\}$
- m is the size of array

Double hashing

Two hash functions $h_1()$ and $h_2()$

Key is k

Size of array data structure = m

- $h_1(k) = k \bmod 13$

- $h_2(k) = 1 + k \bmod 11$

$$h(k, i) = (h_1(k) + i * h_2(k)) \% m$$

Insert 79, 69, 72, 98, 14 using double hashing

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	

Insert $k = 79$

$$h_1(k) = k \bmod 13$$

$$h_1(79) = 79 \bmod 13$$

Insert 79 and 69 using double hashing

0	
1	79
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	

$$h_1(k) = k \bmod 13$$

$$h_1(69) = 69 \bmod 13$$

Insert 79, 69, 72 using double hashing

0	
1	79
2	
3	
4	69
5	
6	
7	72
8	
9	
10	
11	
12	

$$h_1(k) = k \bmod 13$$

$$h_1(72) = 72 \bmod 13$$

Insert 98 using double hashing

0	
1	79
2	
3	
4	69
5	
6	
7	72
8	
9	
10	
11	
12	

$$h_1(k) = k \bmod 13$$

$$h_2(k) = 1 + k \bmod 11$$

$$h_1(98) = 98 \bmod 13 = 7 \quad (\text{collision})$$

2nd hash function

$$h_2(98) = 1 + 98 \bmod 11 = 1 + 10 = 11$$

$$h(k, i) = (h_1(k) + i * h_2(k)) \% m$$

$$\begin{aligned} h(k, 1) &= (7 + 1 * 11) \% 13 \\ &= 18 \% 13 = 5 \end{aligned}$$

Insert k = 98 using double hashing

0	
1	79
2	
3	
4	69
5	98
6	
7	72
8	
9	
10	
11	
12	

$$h_1(k) = k \bmod 13$$

$$h_2(k) = 1 + k \bmod 11$$

$$h_1(98) = 98 \bmod 13 = 7 \quad \text{Collision}$$

$$h_2(98) = 1 + 98 \bmod 11 = 1 + 10 = 11$$

$$h(k, i) = (h_1(k) + i * h_2(k)) \% m$$

$$h(k, 1) = (7 + 1 * 11) \% 13$$

$$= 18 \% 13$$

$$= 5$$

Collision resolution by double hashing

Double hashing

$$h_1(k) = k \bmod 13$$

$$h_2(k) = 1 + k \bmod 11$$

Find if element k is in the hash table

1) Search in index given by $h_1(k)$

2) If collision then

evaluate $h_2(k)$

search in index = $h_1(k) + i * h_2(k)$

3) Use modulo on index
% with size of array m

Insert 14 by double hashing

$$h_1(14) = 14 \bmod 13 = 1$$

$$h_2(14) = 1 + 14 \bmod 11 = 4$$

$$\begin{aligned} h(14, 1) &= h_1(14) + i * h_2(14) \\ &= (1 + 1 * 4) \% 13 \\ &= 5 \% 13 = 5 \end{aligned}$$

$$\begin{aligned} h(14, 2) &= (1 + 2 * 4) \% 13 \\ &= 9 \% 13 = 9 \end{aligned}$$

0	
1	79
2	
3	
4	69
5	98
6	
7	72
8	
9	14
10	
11	50
12	

$$h(k, i) = (h_1(k) + i * h_2(k)) \% m$$

Figure 11.5 Insertion by double hashing. Here we have a hash table of size 13 with $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. Since $14 \equiv 1 \pmod{13}$ and $14 \equiv 3 \pmod{11}$, we insert the key 14 into empty slot 9, after examining slots 1 and 5 and finding them to be occupied.

Insert 14 leads to two collisions at index 1 and 5

$$h_1(14) = 14 \bmod 13 = 1$$

$$h_2(14) = 1 + 14 \bmod 11 = 4$$

$$\begin{aligned} h(14, 1) &= h_1(14) + i * h_2(14) \\ &= (1 + 1 * 4) \% 13 \\ &= 5 \% 13 = 5 \end{aligned}$$

$$\begin{aligned} h(14, 2) &= (1 + 2 * 4) \% 13 \\ &= 9 \% 13 = 9 \end{aligned}$$

0	
1	79
2	
3	
4	69
5	98
6	
7	72
8	
9	14
10	
11	50
12	

$$h(k, i) = (h_1(k) + i * h_2(k)) \% m$$

Figure 11.5 Insertion by double hashing. Here we have a hash table of size 13 with $h_1(k) = k \bmod 13$ and $h_2(k) = 1 + (k \bmod 11)$. Since $14 \equiv 1 \pmod{13}$ and $14 \equiv 3 \pmod{11}$, we insert the key 14 into empty slot 9, after examining slots 1 and 5 and finding them to be occupied.

Cuckoo Hashing

- Cuckoo Hijacks Warbler Nest | Natural World | BBC Earth
<https://www.youtube.com/watch?v=dAU5MTXmAPY>

Common Cuckoo chick ejects eggs of Reed Warbler out of the nest

Need for faster lookups

High-speed routers and switches need to look up IP addresses or flow rules in nanoseconds.

Guaranteed $O(1)$ lookup

Collision resolution by Cuckoo hashing

- Use two hash tables with hash functions $h_1(k)$ and $h_2(k)$

$h_1(k)$ 

$h_2(k)$ 

Key can be stored in one of the two tables

Look up in a Cuckoo hashtable

```
def lookup(key):  
    # Check the first possible location  
    pos1 = h1(key)  
    if table1[pos1] == key:  
        return value1[pos1]  
  
    # Check the second possible location  
    pos2 = h2(key)  
    if table2[pos2] == key:  
        return value2[pos2]  
  
    # Key not found in either location  
    return NOT_FOUND
```

Collision resolution by Cuckoo hashing

- Search is fast because element is checked in two locations only.
Worst-case constant lookup time
- If there is a collision in one table,
move the older element to new location in the other table
to make room for current element (cuckoo behavior)

Cuckoo hashing

Input: {20, 50, 53, 75, 100, 67}

$$h_1(\text{key}) = \text{key} \% 11$$

$$h_2(\text{key}) = (\text{key} / 11) \% 11$$

	20	50	53	75	100	67
$h_1(\text{key})$	9	6	9	9		
$h_2(\text{key})$	1	4	4	6		

Insert 20

Index	0	1	2	3	4	5	6	7	8	9	10
Array[1]										20	
Array[2]											

$$h_1(20) = 20 \% 11 = 9$$

Insert 50

Index	0	1	2	3	4	5	6	7	8	9	10
Array[1]							50			20	
Array[2]											

key	20	50	53	75
$h_1(\text{key})$	9	6	9	9
$h_2(\text{key})$	1	4	4	6

Insert 53

Index	0	1	2	3	4	5	6	7	8	9	10
Array[1]							50			53	
Array[2]		20									

key	20	50	53	75
$h_1(\text{key})$	9	6	9	9
$h_2(\text{key})$	1	4	4	6

Insert 75

Index	0	1	2	3	4	5	6	7	8	9	10
Array[1]							50			75	
Array[2]		20			53						

key	20	50	53	75
$h_1(\text{key})$	9	6	9	9
$h_2(\text{key})$	1	4	4	6

Cuckoo Hashing Quiz: Insert 100

Index	0	1	2	3	4	5	6	7	8	9	10
Array[1]							50			75	
Array[2]		20			53						

key	20	50	53	75	100	
$h_1(\text{key})$	9	6	9	9	1	
$h_2(\text{key})$	1	4	4	6	9	

Cuckoo Hashing Quiz: Insert 67

Index	0	1	2	3	4	5	6	7	8	9	10
Array[1]		100					50			75	
Array[2]		20			53						

key	20	50	53	75	100	67
$h_1(\text{key})$	9	6	9	9	1	1
$h_2(\text{key})$	1	4	4	6	9	6

Insert 67

Index	0	1	2	3	4	5	6	7	8	9	10
Array[1]		67					50			75	
Array[2]		20			53					100	

key	20	50	53	75	100	67
$h_1(\text{key})$	9	6	9	9	1	1
$h_2(\text{key})$	1	4	4	6	9	6

Controlling infinite loop due to collision

- After a MAX number of iterations due to collisions
- Rehash the data to new tables using new hash functions
- Insert(x)
- Load factor kept below 50%

Summary

- Why to use hashing ?
- When to use hashing ?
- How to use hashing
- How to resolve collisions ?
- Double Hashing and Cuckoo Hashing examples.